

TECHNICAL MANUAL

FlowLeads

A Plain-English Guide to How This App Was Built

Written for a first-time founder with no coding background - so you can understand your own product,
and explain it confidently to developers, investors, or partners.

Prepared for: Okezie Collington

Date: August 8, 2026

Project repository: github.com/Collington-1/flowleads

Live app: flowleadsapp.netlify.app

Contents

1. The Big Picture - In Plain English	3
2. What FlowLeads Actually Does	4
3. The Technology Stack	5
4. How a Message Actually Travels (a Real Example)	7
5. The Database - What Gets Stored, and Where	8
6. Where Everything Lives (the Hosting Map)	10
7. Clearing Up the "NextAuth" Confusion	11
8. The Investor One-Pager	12
9. Questions an Investor Might Ask	13
10. Glossary - Every Term, Defined Simply	14

Tip: if you read nothing else, read Part 1 and Part 8. Keep Part 10 (the Glossary) open in a second tab while you read everything else.

The Big Picture - In Plain English

Forget code for a second. Think of FlowLeads as a restaurant.

- **The dining room** - is what your customers see and touch - the menu, the tables, the decor. In FlowLeads, this is the website itself: the screens, buttons, and forms you click on. In coding terms this is called the "frontend."
- **The kitchen** - is where the real work happens, out of sight. Orders come in, food gets made, and plates go back out. In FlowLeads, this is the "backend" - a program running on a server that does the actual work: checking passwords, sending WhatsApp messages, saving data.
- **The pantry / filing cabinet** - is where every ingredient and every past order is stored so nothing is lost. In FlowLeads, this is the "database" - a permanent, organized store of every contact, message, and user account.
- **The waiters** - carry orders between the dining room and the kitchen. In FlowLeads, this carrying-of-information between the website and the backend is called an "API" - a fixed, agreed-upon way for the two sides to talk to each other.
- **The building itself** - FlowLeads doesn't own a physical building - it rents space on other companies' computers, permanently switched on, somewhere in the world. This is called "hosting," and it costs a monthly fee instead of a mortgage.

The one sentence version

IN ONE SENTENCE

FlowLeads is a website (built with React) that talks to a background program (built with Node.js) which saves everything into an online database (Supabase/Postgres) and connects out to WhatsApp and email to send messages on a schedule - and all three pieces run 24/7 on rented cloud computers, not on your own phone or laptop.

Why it's built as three separate pieces

Almost every modern web app - Instagram, your banking app, Gmail - is split the same way: one part the user sees (frontend), one part that does the work in private (backend), and one part that remembers everything (database). Splitting it up this way means each piece can be updated, scaled, or replaced on its own without breaking the others. This is a completely standard, unremarkable architecture - it's not a shortcut or a hack, it's how professional software is built.

What FlowLeads Actually Does

Strip away the tech and FlowLeads is a WhatsApp + Email CRM (Customer Relationship Management tool) for small businesses. Here is what a paying customer can actually do inside it:

- **Connect WhatsApp** - link up to 3 WhatsApp numbers to their account by scanning a QR code - the same way WhatsApp Web works.
- **Chat inside the CRM** - read and reply to WhatsApp conversations from inside FlowLeads instead of their phone.
- **Store contacts** - keep a searchable list of leads/customers with phone numbers, emails, tags, and a deal stage (New !' Contacted !' Qualified !' Won/Lost).
- **Send mass WhatsApp campaigns** - broadcast a message to a group of contacts at once, with built-in pacing so WhatsApp doesn't flag the number as spam.
- **Send email campaigns** - connect their own email account and send email blasts the same way.
- **Schedule messages** - queue a 1:1 message or a full campaign to go out at a future date and time, automatically.
- **Invite teammates** - a business owner can invite staff to share the same CRM under one team account.
- **Import contacts in bulk** - upload a CSV file of contacts instead of adding them one by one, with duplicate-number detection.

Who FlowLeads is for

Small and mid-sized businesses - especially in markets like Nigeria, where WhatsApp is the primary way businesses talk to customers - who currently manage sales conversations manually on a personal phone and lose track of leads. FlowLeads turns that phone-based chaos into an organized, shared, trackable system.

The Technology Stack

This is the part an investor or developer will actually ask about. Below is every piece of technology used, grouped by what it's responsible for, in plain English.

3.1 - Frontend (what the user sees, in the browser)

Technology	What it is	Its job in FlowLeads
React	A JavaScript library for building interactive screens	Builds every page and button in the app (Contacts, Chats, Templates, etc.)
Vite	A build tool	Packages the React code into a small, fast-loading website
Tailwind CSS	A styling toolkit	Controls colors, spacing, and layout - like a box of pre-made design pieces
React Router	A navigation library	Switches between pages instantly without reloading the whole site

3.2 - Backend (the engine, running behind the scenes)

Technology	What it is	Its job in FlowLeads
Node.js	A JavaScript runtime for servers	Runs the backend program 24/7 on a rented cloud server
Express	A backend framework	Organizes the backend into "routes" like /contacts or /admin/ users
Baileys	An open-source WhatsApp connector	Links to WhatsApp the same way WhatsApp Web does, no official Meta API needed
Nodemailer	An email-sending library	Sends email campaigns through each customer's own connected email account
Resend	A transactional email service	Sends FlowLeads' own internal alerts (e.g. "someone new just signed up")

3.3 - Database & Accounts (permanent storage and logins)

Technology	What it is	Its job in FlowLeads
Supabase	An all-in-one backend platform	Provides the database, login system, file storage, and live updates
PostgreSQL	The database engine inside Supabase	The actual "filing cabinet" - an industry-standard system for storing structured data in tables

Technology	What it is	Its job in FlowLeads
Supabase Auth	The login/account system	Handles sign-up, passwords, and keeping a user logged in - this is what checks "who are you?"
Supabase Storage	A file storage bucket	Stores uploaded images and campaign media files
Supabase Realtime	A live-update feed	Pushes new WhatsApp messages to the screen instantly, without refreshing

3.4 - Where the code actually runs (hosting)

Service	Hosts	Why it's used
Netlify	The frontend website	A hosting service specialized in fast, simple websites like React apps
Render	The backend engine	A hosting service for programs that need to run continuously in the background
Supabase Cloud	The database itself	Supabase runs the Postgres database on its own servers so nobody has to manage a physical server
GitHub	The source code	Stores every version of the code ever written, like a permanent, searchable history

PLAIN ENGLISH

None of these are exotic or custom-built from scratch. Every one of these tools is a widely-used, industry-standard product that thousands of other startups also run on. That's a good thing to say to an investor: the foundation is proven, not experimental.

How a Message Actually Travels

Here is a real example, followed step by step, showing every technology from Part 3 actually working together. Scenario: you schedule a WhatsApp campaign to 200 contacts, to go out tomorrow at 9:00am.

1. You click "Schedule"

Your browser (running the React frontend, hosted on Netlify) sends your campaign details over the internet to the backend server.

2. The backend saves it

The Node.js/Express backend (hosted on Render) receives the request and writes a new row into the "campaigns" table in the database (Supabase/Postgres), including your chosen send time.

3. The backend keeps watching

A small piece of code inside the backend checks the database every 60 seconds, asking: "is anything scheduled to send right now?" This runs constantly, with nobody watching it.

4. 9:00am arrives

The backend finds your campaign is due. For each of the 200 contacts, it sends a WhatsApp message through your connected number (via Baileys), deliberately spacing the messages out over time so WhatsApp doesn't flag the number as spam.

5. Results get recorded

Every message sent - or any that failed - gets written back into the database, so you can see delivery status inside the Mass Messaging dashboard.

THIS ANSWERS A COMMON QUESTION

Because Steps 3-5 all happen inside the backend on Render's cloud servers - not on your phone or laptop - a scheduled message will still send even if your own device is switched off. The only thing that could interrupt it is the backend server itself going offline (for example, a free hosting plan "sleeping" after inactivity, which is worth upgrading away from once the business is live).

The Database - What Gets Stored, Where

The database is organized into "tables" - think of each table as its own labeled drawer in the filing cabinet, holding one kind of record. FlowLeads currently has 14 tables, grouped below by what they're for.

Accounts & Teams

Table	What it stores
teams	One row per business/customer account (a "team" is a paying business using FlowLeads)
users	One row per person who can log in - linked to a team, with a role (owner/staff)
team_invites	Pending invitations for teammates who haven't joined yet

Contacts & CRM

Table	What it stores
contacts	Every lead/customer: name, phone, email, tags, and deal stage
contact_groups	Named groups of contacts (used to target a campaign at a subset of contacts)
contact_group_members	Which contacts belong to which group

Messaging (WhatsApp + Email)

Table	What it stores
whatsapp_sessions	Each connected WhatsApp number and its connection status
messages	Every WhatsApp message sent or received, with timestamps
templates	Reusable message templates (e.g. "follow-up" or "welcome" messages)
email_accounts	Each customer's own connected email account, with the password encrypted

Campaigns & Automation

Table	What it stores
campaigns	Each mass-messaging campaign: audience, message, schedule, and status
campaign_logs	A record of every individual message a campaign sent, success or failure
scheduled_messages	Individual 1:1 messages queued to send at a future time

Platform & Integrations

Table	What it stores
webhooks	Configuration for sending FlowLeads events out to other external systems

A quick word on "migrations": every change ever made to this database structure - adding a table, adding a column - is saved as a small, numbered file (0001, 0002, 0003...). This gives a complete, replayable history of how the database grew, the same way GitHub gives a history of the code.

Where Everything Lives (the Hosting Map)

If an investor or developer asks "where does this actually run?" - here is the honest, complete answer.

Piece	Runs on	Address	Who pays
Frontend website	Netlify	flowleadsapp.netlify.app	FlowLeads (hosting bill)
Backend engine	Render	flowleads-backend-wogp.onrender.com	FlowLeads (hosting bill)
Database	Supabase Cloud	A private Supabase project (not public)	FlowLeads (hosting bill)
Source code history	GitHub	github.com/Collington-1/flowleads	Free for a private repo of this size
Internal admin emails	Resend	Used only by the backend, not user-facing	Free tier at current volume

GOOD TO KNOW FOR AN INVESTOR CONVERSATION

Every one of these is a separate company FlowLeads has a running account with, billed monthly, and every one can be scaled up (paid a higher tier) as the number of customers grows - there is no physical server sitting in an office that could break or be stolen.

Clearing Up the "NextAuth" Confusion

You mentioned running into a sentence like: "This determines the NextAuth setup, database schema, Postgres, Render" and not understanding it. Here is the direct answer:

FLOWLEADS DOES NOT USE NEXTAUTH

NextAuth is a login/authentication library built specifically for a different framework called Next.js. FlowLeads is not built with Next.js, and NextAuth does not appear anywhere in this project's code. Whatever produced that sentence was likely describing a different, generic project template - not FlowLeads specifically.

What FlowLeads actually uses for logins

FlowLeads uses Supabase Auth instead - the login system built into Supabase, the same platform that hosts the database. When someone signs up or logs in, Supabase Auth checks their email/password, hands back a secure "session token" (a temporary digital ID card), and the frontend and backend both use that token to confirm who's asking before showing any data.

Why this distinction matters

If you're ever asked "what auth system do you use," the accurate answer is "Supabase Auth" - not NextAuth. Naming the wrong tool to a technical investor or a developer you hire later can cause real confusion, since NextAuth and Supabase Auth are configured completely differently. This manual exists partly so you never have to guess again.

The Investor One-Pager

This page is written so you can read it almost word-for-word, or hand it over as-is.

Product

FlowLeads is a multi-tenant SaaS CRM that lets small businesses manage WhatsApp and email conversations, run mass-messaging campaigns, and track leads through a deal pipeline - built around the fact that most small businesses in markets like Nigeria already run sales through WhatsApp on a personal phone.

Architecture

A standard three-tier web architecture: a React single-page frontend, a Node.js/Express REST API backend, and a managed PostgreSQL database via Supabase. The frontend and backend are deployed independently (Netlify and Render respectively), which is a conventional, horizontally-scalable setup used by the large majority of modern SaaS products.

Key integrations

- **WhatsApp connectivity** - via Baileys, an open-source library implementing WhatsApp's multi-device protocol - avoiding the cost and approval process of the official WhatsApp Business API while functioning equivalently for outbound/inbound messaging.
- **Email** - customers connect their own SMTP account (Gmail, Yahoo, custom domains); platform-level notifications are sent separately via Resend.
- **Authentication & data** - Supabase Auth for login/session management, Postgres row-level security policies to keep every team's data isolated from every other team's.

Multi-tenancy & security

Every business is a "team" in the database, and Postgres row-level security enforces that a team can only ever read or write its own data - this is checked at the database level, not just in application code, which is the stronger of the two approaches. Sensitive credentials (each customer's SMTP password) are encrypted at rest using AES-256-GCM before being stored.

Scalability

Because the frontend, backend, and database are three independently hosted services, each can be scaled up on its own as usage grows (e.g., moving the backend to a larger Render instance, or the database to a larger Supabase tier) without a rebuild or migration of the application itself.

Questions an Investor Might Ask

Q: What's your tech stack?

A: React frontend, Node.js/Express backend, PostgreSQL database via Supabase, hosted on Netlify and Render. All widely-used, industry-standard technology - see Part 3.

Q: Is this scalable?

A: Yes. Each of the three layers (frontend, backend, database) scales independently by upgrading its hosting tier - this is a standard, well-understood pattern, not something that needs to be rebuilt later.

Q: What happens if WhatsApp changes their platform?

A: The WhatsApp connection goes through Baileys, an actively-maintained open-source library that many companies rely on. If WhatsApp materially changed its protocol, the library would need an update - the same risk any product integrating with a third-party platform carries, WhatsApp Business API included.

Q: How is customer data secured?

A: Each business's data is isolated at the database level using Postgres row-level security, so one customer can never read another's data even if there were a bug in the application code. Stored email passwords are encrypted, not plain text.

Q: What are your hosting costs at this stage?

A: Early-stage usage runs on low-cost or free tiers of Netlify, Render, and Supabase - the same providers scale to paid tiers with usage, so costs grow with revenue rather than requiring large upfront infrastructure spend.

Q: Do you own the code, or is it a template?

A: The code is custom-written for FlowLeads and lives in a private GitHub repository with a full commit history showing its development over time.

Glossary - Every Term, Defined Simply

Every technical word used anywhere in this manual (or that you're likely to hear about this project), defined in one or two plain sentences. Alphabetical order - use it like a dictionary.

API (Application Programming Interface)

A fixed, agreed-upon way for two pieces of software to talk to each other - like a waiter carrying a very specific order form between the dining room and the kitchen.

Authentication

Proving who you are (logging in with a password). Different from "authorization," which is deciding what you're allowed to do once logged in.

Backend

The part of an app that runs privately on a server and does the real work - checking passwords, saving data, sending messages - never directly seen by the user.

Build

The process of converting human-written code into the compact, optimized files a browser actually loads.

Cloud / Cloud hosting

Renting computing power from a company (like Netlify, Render, or Supabase) instead of owning and maintaining a physical computer yourself.

Commit

A saved snapshot of code changes in Git, with a short message describing what changed - like saving a Word document with version notes.

Database

A permanent, organized store of information, structured into tables - the digital equivalent of a filing cabinet.

Deploy / Deployment

The act of publishing new code so it becomes the live, running version of the app that real users interact with.

Encryption

Scrambling sensitive information (like a password) so it's unreadable to anyone without the correct key to unscramble it.

Endpoint / Route

A specific web address the backend listens on for a specific purpose - e.g. `/contacts` for contact data, `/admin/users` for admin data.

Environment variable

A secret or setting (like a password or a web address) kept outside the actual code, so it can differ between your computer and the live server without editing code.

Framework

A pre-built structure or toolkit that gives a program a starting shape, so developers don't reinvent basic patterns from scratch (e.g., Express for backends, React for frontends).

Frontend

The part of an app the user directly sees and clicks on - pages, buttons, forms - running inside their web browser.

Git

A system for tracking every change ever made to a project's code over time, so nothing is ever permanently lost and changes can be undone.

GitHub

A website that hosts Git projects online, so code can be backed up, shared, and collaborated on remotely.

JavaScript

The programming language used to build both the frontend (React) and the backend (Node.js) of FlowLeads - one language, used in two different places.

JSON

A simple, text-based format for structuring data (like a labeled list) that different programs - frontend, backend, database - can all read and write easily.

Library

A ready-made piece of code that does one specific job well (e.g., Baileys for WhatsApp, Nodemailer for email), which a developer plugs into their own project instead of writing from scratch.

Migration

A small, numbered file describing one change to the database's structure (like adding a new table or column) - kept in order so the database's full history can be replayed or understood later.

Multi-tenant

One single running app serving many separate customers ("tenants") at once, while keeping each customer's data completely separate from the others.

npm (Node Package Manager)

The tool used to download and install pre-built code libraries (like React or Express) into a JavaScript project.

Open source

Code that is freely published for anyone to view, use, and contribute to - Baileys, React, and Express are all open source.

PostgreSQL (Postgres)

The specific, industry-standard database engine that actually stores FlowLeads' data, running underneath Supabase.

Push

Uploading your locally saved code changes (commits) up to GitHub, making them visible to everyone with access and, often, triggering a deployment.

Query

A request sent to a database asking for specific information - e.g., "get me every contact tagged 'VIP'."

React

A JavaScript library for building interactive, clickable web interfaces out of reusable pieces called "components."

Repository (repo)

A project's complete folder of code plus its entire saved history, usually stored on GitHub.

REST API

A common style of API where the backend exposes a set of fixed web addresses (endpoints), and the frontend asks for or sends data to those addresses.

Row-level security (RLS)

A database-level rule enforced by Postgres itself that restricts which rows of data a given user is allowed to see or change - used here to keep each business's data private from every other business.

SaaS (Software as a Service)

A business model where customers pay an ongoing subscription to use software that runs on the provider's own servers, rather than buying and installing it themselves.

Server

A computer (usually rented, in the cloud) that stays switched on and runs a program continuously so it can respond to requests at any time.

Session

A period of being "logged in" - tracked using a token so the app doesn't ask for a password on every single click.

SMTP

The standard protocol used to send email; FlowLeads lets each customer connect their own SMTP-enabled email account to send campaigns.

Token

A temporary, secret string of characters that proves a user is logged in - like a wristband handed out at the door of an event, checked instead of your ID every time.

Webhook

A way for one system to automatically notify another system the moment something happens, by sending it a message over the internet - instead of the second system having to keep asking "did anything happen yet?"

